EDURA - Learning Management System

Final Development Document

By

W.G.N.DANANJAYA
SE/2021/047
R.W.V.I.D.RAJAPAKSHA
SE/2021/029
P.D.D.RANSIKA
SE/2021/034
H.T.MADUSHANKA
SE/2021/011

A report submitted in partial fulfillment of the requirements for the degree of Bachelor of Science Honors in Software Engineering (B.Sc. SE)

Software Engineering Teaching Unit

Faculty of Science

University of Kelaniya

Sri Lanka

2026

Table Of Contents

Chapter 1: Introduction 2

1.1 Client Details 3

1.2 Problem Statement 3

1.3 Proposed Solution Overview 3

Architectural Note: Database Revision 4

1.4 Stakeholders 4

1.5 Development Continuation from SENG 31242 5

Chapter 2: System Analysis 5

2.1 Purpose and Scope 6

2.2 Business Objectives and Stakeholders 6

2.3 Functional Requirements 7

2.4 Non-Functional Requirements 8

2.5 Business Rules 8

2.6 Constraints and Assumptions 9

2.6.1 Constraints 9

2.6.2 Assumptions 9

2.7 Use Cases 9

2.8 Alternative Solutions Considered 11

2.9 Feasibility Study and Risk Analysis 11

2.9.1 Feasibility 11

2.9.2 Risk Analysis 12

2.10 Scope Changes Since SRS v1.0 12

Chapter 3: System Design 14

3.1 Architectural Overview 15

3.2 Service Decomposition 16

3.3 Inter-Service Communication 16

3.3.1 Event Catalogue 17

3.3.2 Queues and Consumers 17

3.3.3 Local Replicas 18

3.4 Database Design 18
3.4.1 Database Architecture 18
3.4.2 Data Model 18
3.4.3 Normalisation 19
3.4.4 Redis Usage 19
3.5 Software Structure and Class Design 20
3.6 Key Interaction Flows 21
3.7 User Interface Design 23
3.8 Security Design 23
3.9 Deployment Design 24
3.9.1 Repositories and Environments 24
3.9.2 Runtime Topology 24
3.9.3 CI/CD Pipeline 25
3.10 Register of Design Changes from the Approved SDS 26
Chapter 4: System Implementation 27
4.1 Technology Stack 28
4.2 Module and Component Descriptions 30
4.2.1 Auth Service (Port 8001) 30
4.2.2 User Service (Port 8002) 31
4.2.3 Course Service (Port 8003) 31
4.2.4 Content Service (Port 8004) 32
4.2.5 Enrollment Service (Port 8005) 32
4.2.6 Payment Service (Port 8006) 33
4.2.7 Assessment Service (Port 8007) 33
4.2.8 Progress Service (Port 8008) 34
4.2.9 Notification Service (Port 8009) 34
4.2.10 Analytics Service (Port 8010) 34
4.2.11 Admin Service (Port 8011) 35
4.3 Database Implementation 35
4.3.1 Database Architecture 35
4.3.2 ORM Models 35
4.3.3 Migrations 36

4.3.4 Connection Management 36
4.4 CI/CD Pipeline Description 36
4.4.1 Backend CI (edura-backend) 36
4.4.2 Infrastructure CI (edura-infra) 37
4.4.3 Infrastructure as Code 37
4.4.4 Secrets Management 37
4.5 Engineering Challenges and Decisions Made 37
4.5.1 Cloud Cost Optimisation — AKS Deprovisioning (Sprint 5) 38
4.5.2 Python Dependency Standardisation (Sprint 5) 38
4.5.3 Payment Idempotency and Double-Charge Prevention (Sprint 6) 38
4.5.4 Asynchronous Event Debugging (Sprint 6) 38
4.5.5 Notification Service Email Provider Migration (Post Sprint 6) 38
4.5.6 Frontend–Backend Contract Alignment (Sprint 6) 39
Chapter 5: Testing 39
5.1 Testing Strategy 40
5.2 Test Environment 41
5.3 Test Execution Summary 41
5.3.1 Service-Level Tests 41
5.3.2 Cross-Service Integration and Smoke Tests 42
5.4 Test Case Register 42
5.5 Coverage Report Summary 43
5.6 Performance Test Results 44
5.7 Defects Found and Test Limitations 45
5.8 Security Compliance Evidence 46
Chapter 6: Evaluation 46
6.1 Degree of Objectives Met 47
6.2 Client Feedback 49
6.3 Limitations and Known Defects 50
6.4 Future Work 51
Chapter 7: Conclusion 52
Chapter 8: References 53
Appendices A : GitHub Repository Submission 55

Appendix B : Test case register 56

Chapter 1: Introduction

This chapter introduces the EDURA Learning Management System and its development context. It updates the original introduction from the SENG 31242 design report to reflect decisions made during implementation, including final technology choices, the evolved scope of the system, and the formal structure of the development team's engagement with the client.

1.1 Client Details

- Client Name: Dimuthu Prabath
- Organisation: ICT – Dimuthu
- Address: No. 472, Right Bank, Pemaduwa, Anuradhapura, Sri Lanka
- Contact: +94 76 373 5379

ICT – Dimuthu is an Advanced Level ICT tuition institute operating in Anuradhapura, Sri Lanka. The institute provides educational content, assessments, and coaching to A/L students across Sri Lanka. At the time of the initial engagement in SENG 31242, all operationss, including distributing course materials, conducting assessments, and managing student payments were handled manually, limiting the institute's ability to scale and deliver consistent quality education.

1.2 Problem Statement

The existing manual methods for distributing educational materials, administering assessments, and verifying monthly student payments for Advanced Level (A/L) ICT students at ICT – Dimuthu are inefficient and difficult to scale. The absence of a dedicated digital platform results in:

- Course materials shared through informal channels with no structured delivery or access control.
- Assessments conducted manually with no automated grading or progress tracking.
- Student subscription payments collected and verified by hand, creating administrative bottlenecks.
- No centralised dashboard for teachers or administrators to monitor student progress or performance.
- No mechanism for students to revisit past lessons or track their own learning journey.

A dedicated digital platform is required to modernise and streamline these educational and administrative workflows (Pan et al., 2024; Liliasari and Riza, 2023).

1.3 Proposed Solution Overview

EDURA is a cloud-native Learning Management System designed as a B2B custom solution exclusively for ICT – Dimuthu. The system centralises teaching, assessments, and student tracking on one platform, making learning more efficient, engaging, and easy to manage.

Key capabilities of the implemented system include:

- Structured course delivery: Digital course modules and video lessons with YouTube streaming and Cloudinary CDN-backed content distribution.
- Automated assessment: MCQ and True/False quiz execution with Redis-backed timed sessions, automated grading, and anti-cheat tab-switch detection.
- Hybrid payment processing: PayHere payment gateway integration with webhook signature verification (the MD5-based signature that PayHere specifies), plus a manual bank receipt upload and admin verification fallback.
- Event-driven enrollment: RabbitMQ-based messaging ensuring enrollment status is automatically granted or revoked in response to payment and subscription lifecycle events.
- Gamification: Leaderboard caching with Redis sorted sets, progress percentage tracking, and certificate issuance.
- Transactional notifications: Automated email dispatch via the Resend API for verification and password-reset links, payment confirmations, and course certificates.
- Comprehensive analytics: Admin and teacher dashboards with PDF and Excel report exports.
- Role-based access control: A centralised JWT-based require_role decorator shared across all 11 microservices.

Architectural Note: Database Revision

As formally documented in the design report (§2.2), the primary database technology was revised from MongoDB to PostgreSQL (Neon) prior to implementation. This decision was driven by the need for relational integrity, strong ACID transactional guarantees, and the well-supported SQLAlchemy ORM ecosystem for Python microservices (Kleppmann, 2017).

1.4 Stakeholders

The following stakeholder groups were identified and engaged throughout the project:

1. Client and System Owner: Dimuthu Prabath (ICT – Dimuthu), the primary client, responsible for providing domain knowledge, validating requirements, reviewing deliverables, and providing formal sign-off.
2. End Users:
    - Students: A/L ICT students who access course content, take assessments, track progress, and manage subscriptions.
    - Teachers: Educators who create and manage modules, lessons, and assessments, and who monitor student performance.
    - Administrators: Institute staff who manage user accounts, subscriptions, payments, and overall system configuration.
3. Development Team:
    - W.G.N. Dananjaya (SE/2021/047) - Integration Testing, Auth Service, Enrollment Service, Admin Service
    - R.W.V.I.D. Rajapaksha (SE/2021/029) - Course Service, Content Service, Frontend Integration

- P.D.D. Ransika (SE/2021/034) - Infrastructure, CI/CD, Cloud Deployment
- , Payment Service
- H.T. Madushanka (SE/2021/011) - Assessment Service, Progress Service, Notification Service, Analytics Service

4. Academic Evaluators: Supervisors and assessors from the Software Engineering Teaching Unit, Faculty of Science, University of Kelaniya, who evaluate the project against academic requirements.

1.5 Development Continuation from SENG 31242

This course continues directly from SENG 31242. The design artefacts produced in that course, the Software Requirements Specification (SRS) and the Software Design Specification (SDS) served as the primary implementation blueprint. All implementation decisions are traceable to those documents. Any significant deviations from the approved SDS were documented as Architectural Decision Records (ADRs) and reviewed with the supervisor within one sprint.

Key continuity decisions made at the start of SENG 34213:

- The existing GitHub Organisation was extended with new repositories (edura-backend, edura-frontend, edura-infra, edura-tests).
- The GitHub Project board was extended with the Development phase, continuing from Sprints 1–4 (SENG 31242) into Sprints 5–8 (SENG 34213).
- Semantic versioning milestones were established: Sprint 5 → v0.1.0, Sprint 6 → v0.2.0, Sprint 7 → v0.3.0, Sprint 8 → v1.0.0.

Chapter 2: System Analysis

This chapter presents the system analysis carried forward from the Software Requirements Specification (SRS v1.0, 18/03/2026) produced in SENG 31242. The requirements, business rules, use cases and feasibility findings are reproduced in condensed form because they remain the baseline against which the implementation is evaluated in Chapter 6. Where the scope changed during development, the change is recorded in Section 2.10 rather than silently edited into the original requirements.

2.1 Purpose and Scope

EDURA is a web-based Learning Management System tailored for ICT – Dimuthu, an Advanced Level (A/L) ICT tuition institute. It serves A/L students aged 15–18, the teachers who author their courses, and the administrators who run the institute. The SRS defined the following capabilities:

- Role-based access control for administrators, teachers and students.
- Course, module and lesson management with embedded private YouTube video, Cloudinary media and supplementary notes.
- Multiple-choice, true/false and short-answer assessments with automatic grading of objective questions.
- Continuous tracking of video-watch completion, quiz performance and course progress.
- Analytics and progress reports exportable as PDF and Excel.
- Secure authentication with email verification and anti-cheating controls for timed assessments.
- Gamification through points, badges and leaderboards, and automatic digital certificates on course completion.
- A hybrid monthly-subscription payment model: automated activation through the PayHere gateway, with manual receipt upload and administrator verification as a fallback.

The SRS explicitly excluded native mobile applications, live video classes, integration with external LMS platforms, and advanced AI features such as automated content generation or personalised learning-path recommendation.

2.2 Business Objectives and Stakeholders

The business objective is to digitalise the institute's educational processes, reducing the manual administration and grading workload of the teaching staff while improving access to, and engagement with, A/L ICT learning resources through a centralised, highly available platform.

The stakeholders (client, end users, development team and academic evaluators) are described in Section 1.4. The development boundary agreed in the SRS places the platform, its deployment and its documentation within the team's responsibility, while the creation of video

lectures, quiz questions and study notes, and the provisioning of the PayHere merchant, YouTube and Cloudinary accounts, remain the client's responsibility.

2.3 Functional Requirements

Each functional requirement was assigned a unique identifier, a priority and the stakeholder from whom it originated.

ID	Requirement	Priority	Source
FR-001	Secure authentication and authorisation with role-based access control for Administrators, Teachers and Students	High	Client
FR-002	User registration and password recovery validated through email verification (e.g. OTP)	High	Client
FR-003	Creation, structuring and modification of lessons, including private YouTube video links, Cloudinary media assets and supplementary notes (Teachers, Administrators)	High	Teachers
FR-004	Creation and configuration of MCQ, True/False and short-answer assessments	High	Teachers
FR-005	Automatic grading of submitted objective assessments immediately on completion	High	Client
FR-006	Manual payment-receipt upload by students (fallback), verified and approved by administrators	High	System Admin
FR-007	Secure redirection of users to the PayHere checkout gateway for automated subscription payment	High	Client
FR-008	Secure receipt, authentication and processing of PayHere payment-status webhooks	High	System Admin
FR-009	Automatic grant of course access on a "Success" payment status, without administrator intervention	High	Students
FR-010	Continuous tracking of student progress: percentage of videos watched, quiz scores and course completion	High	Client
FR-011	Platform navigation and contextual learning support through a Google Gemini AI-powered digital assistant	High	Client
FR-012	Analytics and performance reports for individual students and whole classes, exportable as PDF and Excel	Medium	Teachers
FR-013	Anti-cheating measures (e.g. browser lockdown, tab-switch detection) during timed assessments	Medium	Client
FR-014	Gamification: points, achievement badges and performance-based leaderboards	Medium	Students / Client

FR-015	Automatic generation and issue of digital certificates on successful course completion	Medium	Students

2.4 Non-Functional Requirements

The non-functional requirements were categorised according to ISO/IEC 25010, and each was given a measurable acceptance criterion so that it can be verified during testing.

ID	Category	Requirement and acceptance criterion	Priority
NFR-001	Performance	95% of standard UI interactions processed and rendered in under 2.0 s under normal load	High
NFR-002	Reliability	Minimum 99.9% monthly uptime, excluding scheduled maintenance	High
NFR-003	Reliability	Full database backups executed automatically every 24 hours	High
NFR-004	Reliability	100% of missed or failed PayHere webhooks detected and queued for reconciliation polling within 15 minutes of the transaction timeout	High
NFR-005	Usability	Fully responsive UI across desktop, tablet and mobile, without horizontal scrolling or broken elements	High
NFR-006	Security	100% of passwords hashed with bcrypt; no plain-text storage	High
NFR-007	Security	Email-verification OTPs expire exactly 5 minutes after issue	High
NFR-008	Security	0% of card numbers (PAN) or CVV codes transmitted to or stored in EDURA infrastructure; card processing delegated to PayHere	High
NFR-009	Scalability	At least 500 concurrent user sessions with no more than 20% performance degradation	Medium
NFR-010	Maintainability	Modular REST API architecture with 100% of endpoints documented	Medium
NFR-011	Maintainability	Next.js frontend, Python backend; PEP 8 and ESLint style guidelines followed	High
NFR-012	Portability	Verified on the latest stable Chrome, Firefox, Safari and Edge	Medium

2.5 Business Rules

The following business rules govern system behaviour and must be enforced by the system logic.

ID	Rule
BR-001	Subscription access: course materials and assessments are granted either automatically on a successful PayHere callback or on administrator verification

	of an uploaded receipt. A lapsed subscription automatically suspends access to new course materials.
BR-002	Assessment grading: automatic grading applies only to MCQ and True/False questions; subjective questions are flagged for manual review by teaching staff.
BR-003	Timed examinations: a timed assessment is completed in a single continuous session and is submitted automatically when the time limit expires.
BR-004	Certification eligibility: certificates are issued only after 100% watch status on all mandatory video modules and a minimum passing score in all required assessments.
BR-005	Account integrity: registration, verification and password-recovery OTPs become permanently invalid exactly five minutes after generation. In the implemented system, verification and password recovery use signed emailed links valid for 24 hours and 60 minutes; the five-minute OTP service exists but is not used by the user flows.
BR-006	Anti-cheating enforcement: tab-switch or lockdown violations during timed assessments are recorded and reported to administrators and the relevant teachers.

2.6 Constraints and Assumptions

2.6.1 Constraints

- **Technical:** web-based delivery only; continuous internet connectivity (no offline mode); latest stable versions of the four major browsers; Next.js frontend and Python backend microservices; deployment on Microsoft Azure with containerisation (Docker, and Kubernetes for orchestration); pre-recorded video only.
- **Operational:** video delivery is bound by the terms of service, rate limits and bandwidth allocations of YouTube and Cloudinary; storage capacity is limited by the infrastructure allocated at deployment.
- **Time and resource:** the entire lifecycle is bounded by the academic timetable of the Software Engineering Teaching Unit; all engineering work is performed by the four members of the development team; educational content creation is the client's responsibility.

2.6.2 Assumptions

- Users have foundational digital literacy, a suitable device and a stable broadband connection, and use modern browsers.
- Third-party services (PayHere, YouTube, Cloudinary, the email provider and Microsoft Azure) remain available and meet their published service levels.
- The client supplies all educational content, and provisions a verified PayHere merchant account, a private YouTube channel and a Cloudinary environment before final deployment.
- Administrators review uploaded payment receipts promptly, and students maintain a general standard of academic honesty during unsupervised assessments.

2.7 Use Cases

Three actors interact with the system. The **Student** consumes courses, takes assessments, pays subscriptions and tracks progress. The **Teacher** creates modules, configures assessments and monitors class performance. The **Administrator** manages users, verifies manual payments and oversees the platform. PayHere participates as a secondary (external-system) actor in the automated payment use case.

Five critical use cases were specified in full in the SRS (Chapter 14). They are summarised below, and the use case and activity diagrams for the student, teacher and administrator subsystems are included in the SRS.

Use case	Actor	Main success scenario	Key alternative flows
UC-001 User Authentication and Login	Student, Teacher, Administrator	Credentials are submitted and validated, and the account must be active with a verified email address; access and refresh tokens are issued and a session is created; the user is redirected to the role-based dashboard	Invalid credentials: error and re-entry. Unverified email: the user is asked to verify and a new verification link can be requested
UC-002 Upload Subscription Receipt	Student	The student selects a receipt file (JPEG, PNG or PDF), enters the payment details and submits; the upload is stored and recorded as "Pending Verification"	Invalid file format or size: upload rejected with the accepted formats and limits shown
UC-003 Create Educational Module	Teacher	The teacher enters the module title and description, the private YouTube ID and Cloudinary links and supplementary PDF notes; links are validated and stored; the module is published to active students	Upload interruption: incomplete file discarded and the teacher prompted to restart
UC-004 Attempt Timed Assessment	Student (active subscription)	The student starts the quiz; lockdown and the countdown begin; answers are submitted, auto-graded and the score is recorded and displayed	First tab switch: warning; repeat offences: forced submission and review flag. Time expiry: auto-submit with the selected answers
UC-005 Automated Subscription	Student; PayHere (secondary)	The student chooses "Pay via PayHere" and is redirected to the gateway; payment is authorised; a	Payment declined: no access granted. Webhook timeout: background reconciliation polls PayHere. User

Payment via PayHere		success webhook reaches EDURA; the subscription becomes "Active" and access is granted	cancellation: no change to subscription status

2.8 Alternative Solutions Considered

Four design alternatives were evaluated during analysis. The decisions below shaped the architecture described in Chapter 3.

Decision area	Alternatives evaluated	Selected approach and justification
Architecture	Monolith vs. decoupled microservices with REST APIs	**Microservices.** A monolith is simpler initially but limits flexibility and complicates scaling and integration; microservices permit independent scaling and deployment, fault isolation and parallel development by four developers
Delivery platform	Native mobile apps vs. responsive web application	**Responsive web application.** Maintaining several native codebases was infeasible within the academic schedule and team capacity; a responsive web app is cross-platform without that overhead
Payment processing	Gateway only vs. manual verification only vs. hybrid	**Hybrid.** PayHere gives immediate access and reduces administrative verification; manual receipt upload is retained so that students without cards or bank accounts are not excluded
Content delivery	Local server storage vs. third-party media platforms	**YouTube (private channel) and Cloudinary.** Local storage would exhaust server bandwidth and capacity under concurrent access; the external platforms provide high availability, adaptive bitrate streaming and preserve server bandwidth

2.9 Feasibility Study and Risk Analysis

2.9.1 Feasibility

Dimension	Finding
Technical	Feasible. Microservices demand disciplined containerisation and DevOps practice, but isolate faults and allow independent scaling. Video bandwidth risk is removed by offloading to YouTube and Cloudinary; PayHere webhooks are a well-documented integration; the chosen stack is mature and the team's proficiency was verified through prototyping
Economic	Feasible. No external development cost (academic project); hosting cost is minimised by third-party media, serverless database scaling and pay-as-you-go Azure pricing; the PayHere integration reduces administrative labour for receipt verification
Operational	Feasible. Hybrid payment keeps the platform accessible to all students; teachers embed existing YouTube and Cloudinary URLs rather than uploading large files; administrators focus only on the manual-payment minority

Schedule	Feasible. Decoupled services permit parallel development (for example Authentication, Course and Payment in parallel), and use of the YouTube and Cloudinary APIs removes the need to build streaming and heavy file-handling logic

2.9.2 Risk Analysis

Category	Risk	Probability / Impact	Mitigation
Architectural	Distributed data inconsistency between services	Medium / High	Robust inter-service communication, eventual-consistency models and comprehensive logging to trace failures
External dependency	Outage of YouTube, Cloudinary or PayHere	Low / High	Error handling and graceful UI degradation; background reconciliation polling of PayHere for pending transactions
Security	Extraction and sharing of private video links or asset URLs	High / High	Domain-restricted YouTube embedding; time-expiring signed Cloudinary URLs
Operational	Administrative backlog of manual receipts near examination periods	Medium / Medium	Promote PayHere as the primary method; provide bulk verification tools for administrators
Technical	Integration complexity across Next.js, Python services, data stores and third-party APIs	Medium / Medium	OpenAPI contracts, integration testing in a production-like staging environment, Docker Compose for environment parity
Schedule	Overrun of the academic deadline	Medium / High	Agile delivery with the core MVP prioritised ahead of secondary features such as gamification

The sprint-level risk register maintained during development (for example RSK-04 on cloud cost, RSK-05 on payment idempotency and RSK-10 on frontend–backend contract alignment) is discussed in Section 4.5.

2.10 Scope Changes Since SRS v1.0

The analysis above is unchanged from SENG 31242 except for the revisions listed below. Each arose from an implementation constraint or from client feedback and is recorded here for traceability.

Area	SRS v1.0 position	Final position	Reason
Primary database	MongoDB mandated; relational databases excluded (SRS §11.1, §17.2)	PostgreSQL on Neon, one database per service; Redis retained for sessions, OTPs, timed assessments and leaderboards	Need for relational integrity, ACID guarantees and the SQLAlchemy/Alembic ecosystem (ADR-02; design report §2.2)
AI assistant (FR-011)	Google Gemini assistant for students; advanced AI excluded	Descoped and recorded as future work (LIM-01). Learner support is provided through a support-chat channel to the administration team	API cost and time constraints
Email delivery	SMTP server for OTP and notification email	Resend transactional email API	Unreliable SMTP configuration in CI, staging and the local ISP context (Section 4.5.5)
Social login	Social login and SSO excluded	Optional Google sign-in added alongside email-and-password authentication, linking to an existing account by email	Convenience for students; account unification avoids duplicate accounts
Login flow	OTP on every login (UC-001)	Email-and-password login issues JWT access and refresh tokens; email verification uses a signed link valid for 24 hours and password reset an emailed single-use link valid for 60 minutes; an OTP service with a 5-minute lifetime exists but is not used by the user flows (BR-005)	Reduced login friction; account-integrity operations use emailed links
Hosting and CI/CD	Azure Kubernetes Service; Azure DevOps	GitHub Actions pipelines across four repositories; AKS manifests retained, staging verified in	Alignment with the course guideline and Azure student-credit limits (Section 4.5.1)

	pipelines (SDS §8.5)	Sprint 5 then deprovisioned to conserve credits	
Additional capabilities	Not in SRS	Class packages, subject streams and grade subjects, video checkpoint questions, badges, feedback collection and a student referral and wallet ledger	Implemented during Sprints 5–7 to support class-based teaching and student engagement

Chapter 3: System Design

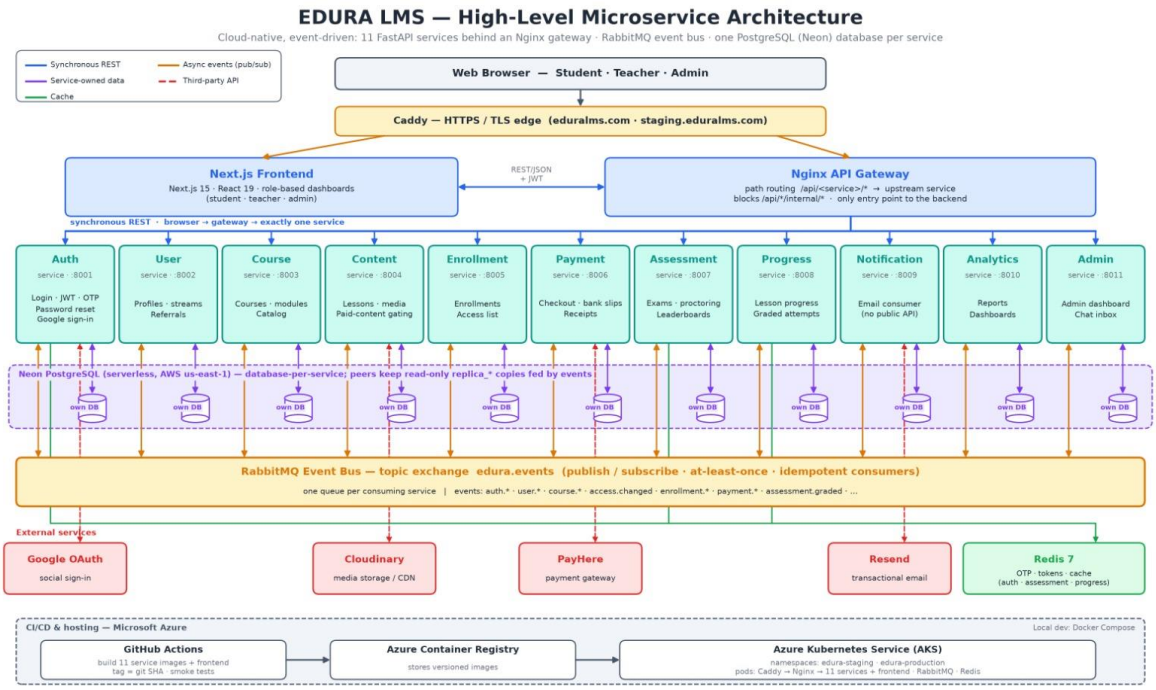
This chapter presents the Software Design Specification (SDS v1.0, 19/04/2026) as updated to reflect the system that was actually implemented. Design elements that remained valid are summarised; elements that changed during development are described in their final form, and every deviation from the approved SDS is collected in the register in Section 3.10.

3.1 Architectural Overview

EDURA follows a microservices architecture with five logical layers:

1. **Client layer:** a Next.js (App Router) and TypeScript single-page application using Redux Toolkit for state management.
2. **Gateway layer:** an Nginx reverse proxy that is the single entry point for all browser traffic. It routes each /api/<service>/ path to its upstream service, applies a rate limit of 100 requests per minute per client address (burst 20, HTTP 429 on excess), sets CORS headers and caps request bodies at 25 MB. In the cloud deployment a Caddy edge proxy terminates HTTPS in front of the gateway.
3. **Service layer:** eleven independent FastAPI services (Section 3.2).
4. **Messaging and cache layer:** a RabbitMQ topic exchange (edura.events) for asynchronous inter-service events, and Redis for OTPs, sessions, timed assessment sessions and leaderboards.
5. **Persistence layer:** PostgreSQL on Neon, with one database per service.

External services are Cloudinary (signed media delivery), YouTube (privately hosted lecture video embedded through youtube-nocookie.com), PayHere (payment gateway), Resend (transactional email) and Google (optional sign-in).



The principal architectural principles are:

- **Database per service.** No service reads another service's database.
- **Event-carried state transfer.** Services do not call each other over HTTP. A service that owns data announces every change as an event; a service that needs the data keeps its own small read-only replica, updated from those events (Section 3.3).
- **Stateless services.** All services are stateless containers, which allows horizontal scaling; shared state lives in PostgreSQL, Redis and RabbitMQ.
- **Shared security library.** Token validation and role enforcement are implemented once in shared/auth.py and imported by all services.

3.2 Service Decomposition

Service	Port	Responsibility	Own database	REST endpoints
Auth	8001	Registration, email verification, login, token refresh and rotation, Google sign-in, password reset, OTP, sessions	auth_db	20
User	8002	Extended profiles, streams, grade subjects, referral and wallet records, role changes	user_db	20
Course	8003	Course, module, lesson, material and video-checkpoint management; publish lifecycle	course_db	25
Content	8004	Signed media URLs, YouTube stream parameters, validated file upload	content_db	7
Enrollment	8005	Enrollment grants and suspensions, lesson access, class packages	enrollment_db	19
Payment	8006	PayHere checkout and webhook, receipt upload and review, bank accounts	payment_db	14
Assessment	8007	Assessments, questions, timed sessions, grading, violations, badges	assessment_db	34
Progress	8008	Lesson and course progress, certificates, leaderboard	progress_db	9
Notification	8009	Event-driven email dispatch and delivery log	notification_db	2
Analytics	8010	Aggregated metrics and PDF/Excel reports	analytics_db	10
Admin	8011	Audit log, feedback, support chat, receipt approval workflow	admin_db	14

3.3 Inter-Service Communication

The SDS allowed both synchronous REST calls and asynchronous messaging. During implementation the team standardised on asynchronous events only, because synchronous calls coupled service availability (for example, content delivery would fail whenever the enrollment service was down).

3.3.1 Event Catalogue

Every routing key on the edura.events topic exchange is defined once in shared/events.py, which is the single source of truth for the broker topology; rabbitmq/setup.py in the infrastructure repository declares the same queues and is run twice in CI to prove it is idempotent.

Producer	Routing keys
Auth	auth.user-registered, auth.account-updated, auth.otp-requested, auth.email-verification-requested, auth.password-reset-requested, auth.password-changed
User	user.profile-upserted, user.profile-deleted, user.stream-upserted, user.stream-deleted
Course	course.upserted, course.deleted, course.published, course.module-upserted, course.module-deleted
Enrollment	enrollment.activated, enrollment.changed, access.changed, subscription.expired
Payment	payment.success, payment.receipt-uploaded, payment.receipt-reviewed, payment.manual-uploaded
Assessment	assessment.upserted, assessment.deleted, assessment.graded, badge.awarded, proctoring.violation, violation.alert
Progress and others	lesson.watched, certificate.issued, report.progress

3.3.2 Queues and Consumers

Each consuming service owns one durable queue bound to the routing keys it needs.

Queue	Consumed events
user.queue	account events; enrollment.activated
course.queue, content.queue	access.changed
enrollment.queue	payment.success, subscription.expired; course, profile and stream replica events
payment.queue	course catalogue events
assessment.queue	access.changed; course, profile and account events
progress.queue	enrollment.activated, assessment.graded, lesson.watched

notification.queue	OTP, verification, password, payment, assessment-graded, certificate and violation events
admin.queue	profile and account events
analytics.queue	account, profile, catalogue, lesson, access, enrollment, payment, receipt, assessment and badge events

3.3.3 Local Replicas

Services keep local replica_* tables (defined in shared/replicas.py) holding the latest state of data owned elsewhere, for example the Analytics Service's replica_enrollments, replica_payments and replica_attempts. Only the event consumer writes to them. This removes runtime coupling between services at the cost of eventual consistency, which is the Saga-style consistency model anticipated by the SRS risk analysis (Section 2.9.2).

3.4 Database Design

3.4.1 Database Architecture

Each service has its own PostgreSQL database on Neon (serverless, with automatic compute scaling). Locally, the eleven databases are created inside a single PostgreSQL 16 container by an initialisation script. Schema changes are managed by Alembic, one migration chain per service.

3.4.2 Data Model

The conceptual entity-relationship diagram of the SDS (students, teachers, administrators, classes, payments and exams) was refined into the following owned tables.

Service	Tables
Auth	users, refresh_tokens, otp_codes
User	user_profiles, streams, grade_subjects, referral_events, wallet_transactions
Course	courses, modules, lessons, materials, video_checkpoints, checkpoint_questions
Content	media access records fed by access.changed
Enrollment	enrollments, lesson_access, class_packages, class_enrollments
Payment	payments, payment_receipts, bank_accounts
Assessment	assessments, questions, submissions, violation_logs, badges, user_badges
Progress	lesson_progress, course_progress, certificates, leaderboard_entries
Notification	notification_logs
Analytics	replica_enrollments, replica_exams, replica_attempts, replica_badges, replica_payments, replica_receipts
Admin	audit_logs, feedbacks, chat_conversations, chat_messages

Key modelling decisions follow the SRS data requirements (Section 17.1):

- A lesson stores the youtube_video_id and cloudinary_asset_url rather than video files.

- A payment records payment_method (MANUAL or PAYHERE), the gateway transaction_id and a payment_status (PENDING, SUCCESS or FAILED).
- Refresh tokens are stored only as SHA-256 hashes, and passwords only as bcrypt hashes.
- Cross-service references (for example a user_id on an enrollment) are plain integers with no database foreign key, since the referenced row lives in another database.



3.4.3 Normalisation

All schemas were designed to third normal form (3NF). Deliberate denormalisation is limited to read-heavy paths: the replica tables above, cached progress summaries, and the Redis leaderboard.

3.4.4 Redis Usage

Purpose	Mechanism
OTP storage	Key with a 300-second TTL, deleted on successful verification (BR-005)
Sessions	Per-user session keys with a 7-day TTL; at most 3 concurrent sessions, oldest evicted
Email-verification resend throttle	Short-lived key preventing repeated resend requests
Timed assessments	Session key whose TTL equals the quiz time limit (BR-003)
Leaderboard	Sorted sets updated with ZADD / ZINCRBY

Analytics summaries	Cached query results for dashboard responsiveness

3.5 Software Structure and Class Design

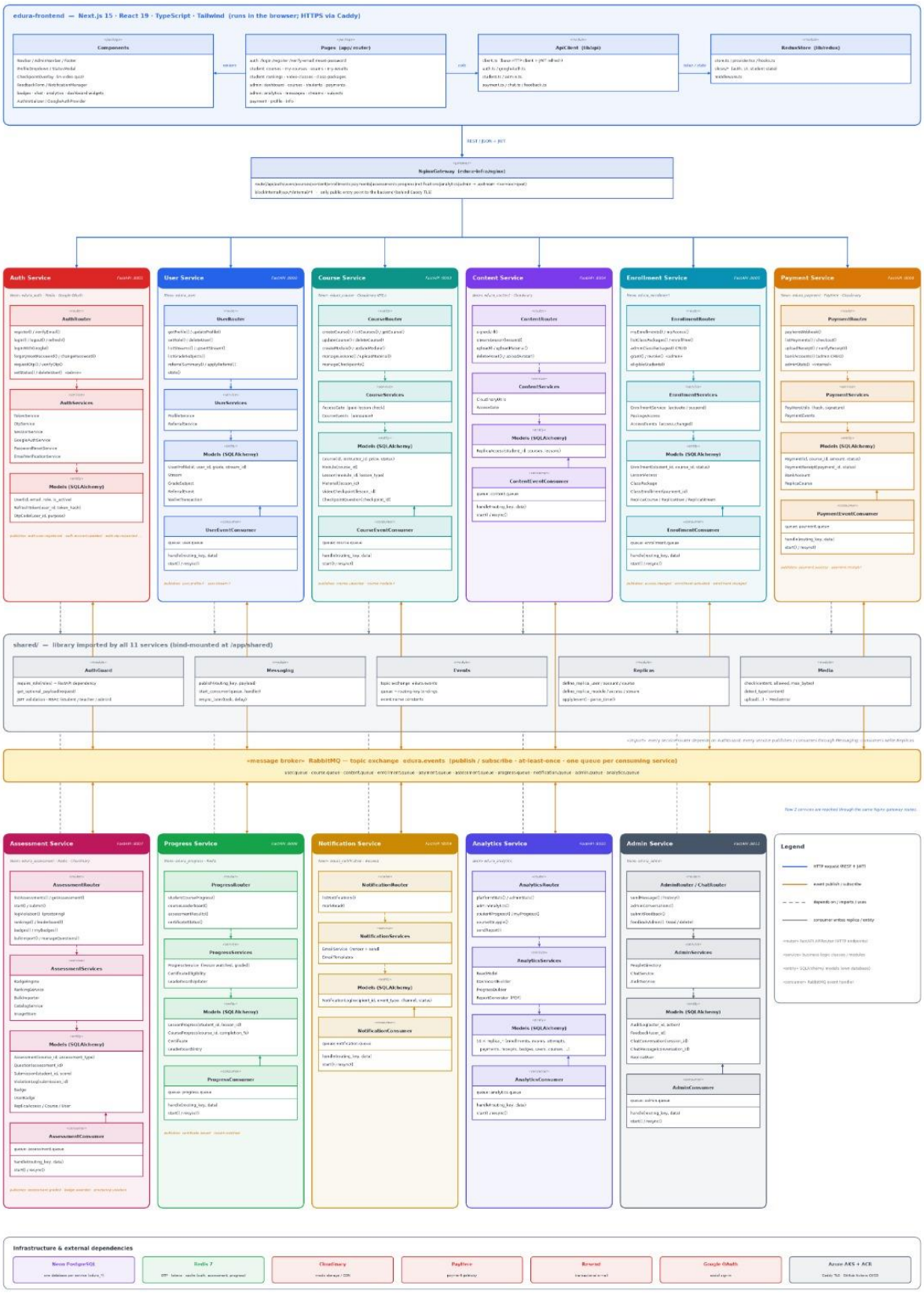
The SDS applied Domain-Driven Design with a layered structure inside each service. The implementation realises this with the same module layout in every service:

Module	Layer and responsibility
main.py	Application composition: FastAPI app, router registration, start-up (including the event consumer)
router.py	Presentation layer: HTTP endpoints, request parsing, role enforcement with Depends(require_role([...]))
schemas.py	Pydantic v2 request and response models; validation of all inbound data
services.py	Business logic, such as TokenService, OtpService and SessionService in the Auth Service
models.py	SQLAlchemy 2.0 domain entities and enumerations
database.py, config.py	Engine and session factory (get_db() dependency); environment-based settings
event_consumer.py	RabbitMQ consumer applying incoming events
alembic/	Migration history

The shared/ package holds the cross-cutting classes and functions: auth.py (JWT validation and require_role), events.py (event catalogue), messaging.py (RabbitMQ publisher and consumer helpers), replicas.py (replica tables) and media.py.

EDURA LMS — High-Level Class Diagram (all components)

Frontend → Nginx gateway → 11 FastAPI services (router · service classes · entities · event consumer) · shared library · RabbitMQ event bus. Services never call each other; they exchange events and keep local replica_* copies



3.6 Key Interaction Flows

The behaviour of the principal use cases in the final design is described below. The corresponding sequence diagrams of SDS Chapter 5 remain valid for the user-facing steps, except where noted.

Registration and login. The client posts to POST /auth/register; the Auth Service stores the user and publishes auth.email-verification-requested; the Notification Service emails a signed verification link (valid for 24 hours) through Resend; the link opens the verify-email page, which submits its token to POST /auth/verify-email, and a new link can be requested once every 60 seconds. Password reset works the same way through POST /auth/forgot-password and POST /auth/reset-password, with an emailed single-use link valid for 60 minutes. Login (POST /auth/login) returns a 15-minute access token and sets a 7-day refresh token in an HttpOnly cookie. POST /auth/refresh rotates the refresh token, and re-use of a rotated token is treated as theft and revokes the sessions.

Course creation and publishing. A teacher creates a course in DRAFT state, adds modules and lessons, and publishes it; publishing requires at least one module containing a lesson. The Course Service then emits course.upserted and course.published, which the Enrollment, Payment, Assessment and Analytics services use to update their replicas.

Payment and enrollment. For PayHere payments, the Payment Service issues a signed checkout request; PayHere calls the webhook, whose signature is verified and which is processed idempotently on transaction_id; the service publishes payment.success. For manual payments, the student uploads a receipt, an administrator approves it, and the same payment.success event is published. The Enrollment Service consumes it, activates or extends the enrollment by 30 days and publishes enrollment.activated and access.changed. The Content, Course and Assessment services update their access data from access.changed; the Progress Service opens a progress record; the Notification Service sends the confirmation email. When a subscription lapses, subscription.expired suspends the enrollment.

Assessment and grading. The student starts an assessment: a Redis session with a TTL equal to the time limit is created, the questions are shuffled and answer keys are withheld. Tab-switch and focus-loss events are posted to the violations endpoint and recorded in violation_logs. On submission, or automatically when the timer expires, MCQ and True/False answers are graded and short answers are flagged for manual review (BR-002). The service publishes assessment.graded.

Progress and certification. The Progress Service consumes lesson.watched and assessment.graded, recalculates course completion, updates the Redis leaderboard and, when the completion and passing-score conditions of BR-004 are met, issues a certificate and publishes certificate.issued for email delivery.

Secure content delivery. To view a lesson, the client requests a signed Cloudinary URL (valid for one hour) or YouTube stream parameters from the Content Service. For students, the

service checks its local access data (kept current by access.changed) and refuses the request if the student has no active enrollment.

Analytics report generation. The Analytics Service builds reports from its local replicas, caches summaries in Redis, and renders PDF reports with ReportLab. Excel export (openpyxl) was part of the design but is not implemented.

AI assistant interaction. SDS Section 5.2.7 specified a Gemini-based assistant. This flow was descoped (Section 3.10).

3.7 User Interface Design

The wireframes and user-flow diagrams of SDS Chapter 6 (authentication, student dashboard, course catalogue, content viewer, assessment, progress and admin dashboards, profile) were used as the specification for the Next.js frontend. The implemented route structure is:

Role	Implemented areas
Public and authentication	Landing page, information pages, login, register, verify-email, forgot-password, reset-password
Student	courses, video-courses, video-classes, class-packages, my-courses, exams, my-results, rankings, profile, feedback; payment for checkout and receipt upload
Administrator	dashboard, analytics, courses, students, admins, payments, class-packages, streams, subjects, rankings, messages, settings

The accessibility targets of the SDS (WCAG 2.1 Level AA: sufficient colour contrast, keyboard operation, visible focus, labelled forms, semantic HTML) continue to apply. Responsive layout is provided with Tailwind CSS, with formal small-screen verification tracked as LIM-06 (Section 6.3).

3.8 Security Design

The defence-in-depth strategy of SDS Chapter 7 was implemented as follows.

Concern	Final design
Authentication	Email and password with bcrypt hashing; optional Google sign-in (POST /auth/google) that links to an existing account by email; email verification through a signed link valid for 24 hours and password reset through an emailed single-use link valid for 60 minutes
Tokens	JWT signed with HS256 by the Auth Service; 15-minute access token; 7-day refresh token in an HttpOnly cookie, stored hashed, rotated on every use with reuse detection
Session control	Redis-backed sessions with a limit of 3 per user; sessions revoked on logout and password change
Authorisation	require_role([...]) in shared/auth.py validates the bearer token and enforces the Student, Teacher or Administrator role in every service, returning 401 (MISSING_TOKEN, TOKEN_EXPIRED, INVALID_TOKEN) or 403 (INSUFFICIENT_PERMISSIONS);

	ownership checks (for example assert_course_owner) restrict teachers to their own courses
Input validation	Pydantic v2 schemas on every endpoint; SQLAlchemy parameterised queries (no string-built SQL); file uploads limited by size, MIME type and magic-number signature (PDF, PNG, JPEG, up to 25 MB)
Payment security	No card data reaches EDURA (NFR-008); PayHere webhook signature verified with constant-time comparison; idempotency on transaction_id
Media protection	Cloudinary signed URLs expiring after one hour; YouTube embedded in privacy-enhanced mode
Transport security	HTTPS terminated at the Caddy edge with automatically renewed Let's Encrypt certificates; gateway rate limiting and CORS policy
Secrets	GitHub environment secrets in CI/CD, injected into Kubernetes secrets at deploy time; .env files excluded from version control and .env.example templates committed
Dependency security	pip-audit and npm audit gates in CI; the Next.js 15 upgrade (ADR-11) resolved high-severity advisories

3.9 Deployment Design

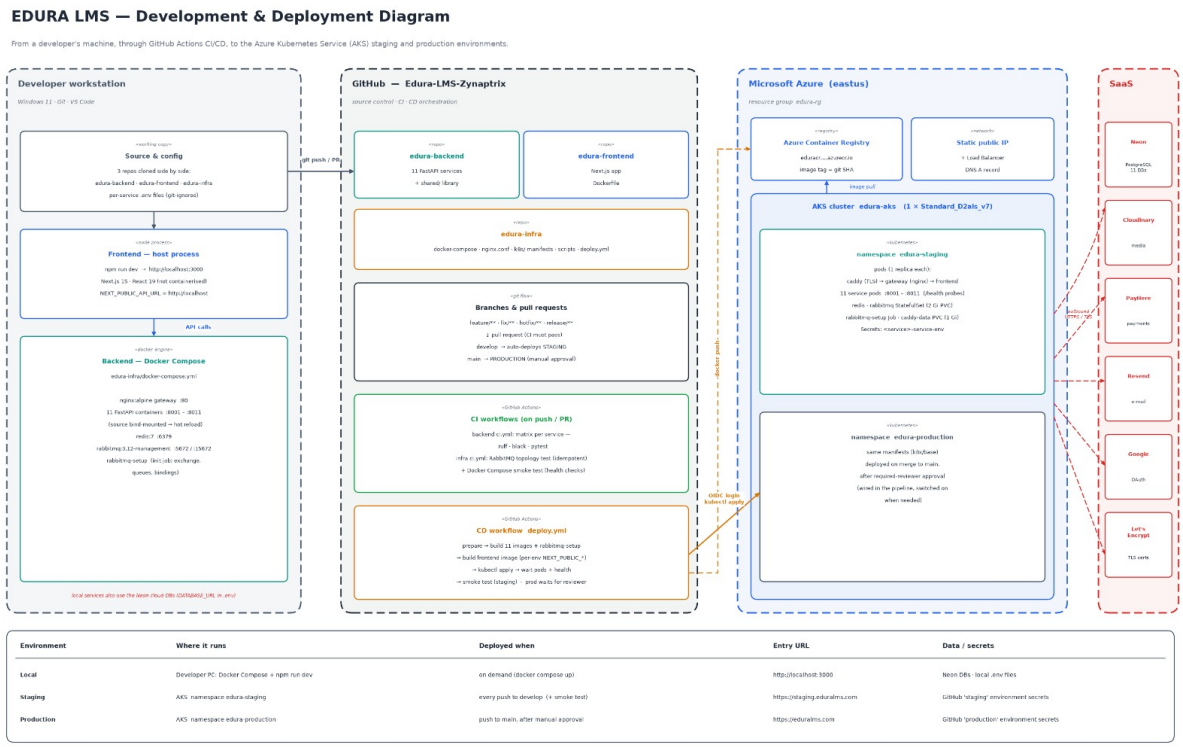
3.9.1 Repositories and Environments

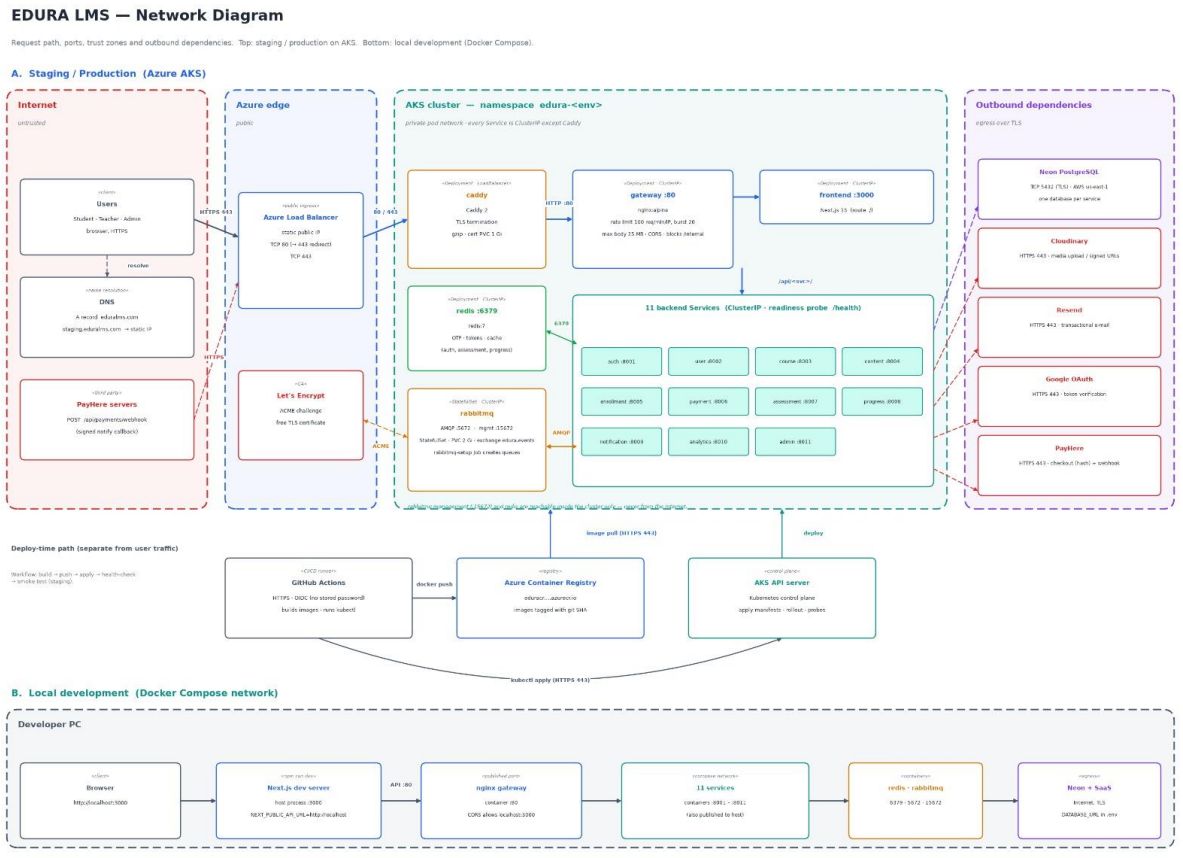
The code base is split into four repositories in the GitHub organisation, as required by the course guideline: edura-backend (eleven services and the shared library), edura-frontend, edura-infra (Docker Compose, Kubernetes manifests, Nginx, RabbitMQ and PostgreSQL scripts, deployment workflows) and edura-tests (integration and smoke tests), alongside the documents repository.

Environment	Description
Local development	Docker Compose stack of the eleven services, PostgreSQL 16, Redis 7, RabbitMQ 3.12 and Nginx; the frontend runs separately through npm run dev
Staging	Azure Kubernetes Service, deployed on merge to develop; verified in Sprint 5 and then deprovisioned to conserve student credits
Production	Azure Kubernetes Service, deployed on merge to main after manual approval

3.9.2 Runtime Topology

On AKS, each backend service runs as a Deployment with a ClusterIP Service. Redis and RabbitMQ run as in-cluster workloads (Redis replaces the SDS's Azure Cache for Redis, to reduce cost). An Nginx gateway routes API traffic, and a Caddy deployment with a persistent volume for certificates exposes the public endpoint through a static IP address. Container images are built per service, tagged with the commit SHA and stored in Azure Container Registry. PostgreSQL remains on Neon.





3.9.3 CI/CD Pipeline

The pipeline is implemented with GitHub Actions (details in Section 4.4): a parallel matrix job per backend service (Ruff and Black, pip-audit, pytest with coverage, Alembic migration cycle); frontend lint, build and npm audit; infrastructure checks (RabbitMQ topology

idempotency and a Docker Compose smoke test); and deployment workflows for staging and production followed by a smoke test.

3.10 Register of Design Changes from the Approved SDS

#	SDS design (v1.0)	Final design	Reason
1	Identity provider: Asgardeo (OAuth 2.0 / OIDC), RS256 tokens validated at the gateway	Auth Service issues HS256 JWTs; every service validates tokens through shared/auth.py; optional Google sign-in	Reduced external dependency and uniform enforcement of roles inside each service
2	Synchronous REST plus asynchronous messaging	Asynchronous events only, with local replicas	Removes runtime coupling and keeps services available when a peer is down
3	Gateway performs authentication-token validation and SSL termination	Gateway routes, rate-limits and sets CORS; Caddy terminates TLS; services authenticate requests	Separation of concerns; automatic certificate management
4	Azure Cache for Redis	Redis running inside the cluster	Cost
5	Azure DevOps CI/CD with canary production deployment	GitHub Actions, with manual approval before production	Course guideline and repository hosting on GitHub
6	Pylint, Snyk and Trivy scanning	Ruff, Black, pip-audit and npm audit	Tooling aligned to the Python 3.12 and Node toolchains used
7	Google Gemini AI assistant (SDS 5.2.7, Figure 10)	Descoped; support chat to administrators provided instead	API cost and time (LIM-01)
8	Email and SMS gateway	Resend email API; SMS not implemented	Reliable email delivery; SMS outside the delivered scope
9	Next.js 14	Next.js 15 with React 19 (ADR-11)	High-severity vulnerabilities in Next.js 14 failed the CI security gate
10	MongoDB, if used (SRS)	PostgreSQL on Neon throughout (ADR-02)	Relational integrity and ACID guarantees
11	One-time code validity (SDS 7.2.4)	Verification link 24 hours; password-reset link 60 minutes; OTP service 5 minutes (not used by the user flows)	Verification and reset are completed by following an emailed link; this replaces the 10-minute code of the

			SDS and the 5-minute code of SRS NFR-007 and BR-005
12	Field-level AES-256-GCM encryption, mutual TLS between services, virus scanning of uploads	Not implemented: passwords and refresh tokens hashed; database encryption at rest provided by Neon; upload validation by size, MIME type and magic number	Prioritised higher-risk controls within the schedule; recorded as future hardening

Chapter 4: System Implementation

This chapter documents the concrete implementation of the EDURA Learning Management System. It covers the final technology stack with justifications, the description of all microservice components with representative code excerpts, the database implementation strategy, the CI/CD pipeline, and the key engineering challenges and decisions encountered during development.

4.1 Technology Stack

The following technology choices were finalised at the start of Sprint 5 and committed to the team's coding standards document.

Layer	Technology	Version	Justification
Frontend Framework	Next.js	15.x	App Router, React Server Components, file-based routing, and strong TypeScript support
Frontend Language	TypeScript	5.x	Type safety for API contracts; strict mode enforced via tsconfig.json
Frontend State	Redux Toolkit	2.x	Predictable state container for authentication and course session slices
Frontend Styling	Tailwind CSS	3.x	Utility-first approach reducing CSS footprint; component variants via class composition
Frontend Animation	Framer Motion	12.x	Declarative animation library for smooth page transitions and gamification feedback
Backend Framework	FastAPI (Python)	0.115.x	ASGI framework with native Pydantic v2 integration, automatic OpenAPI docs, and async support
Backend Language	Python	3.12	Team proficiency; broad ecosystem (SQLAlchemy, Pydantic, PyJWT)
ORM	SQLAlchemy	2.x	Mapped column syntax, relationship management, and session dependency injection
Database Migrations	Alembic	1.x	Per-service migration revisions with automated upgrade/downgrade cycle testing

Primary Database	PostgreSQL (Neon)	16	Serverless, cloud-managed PostgreSQL with per-service isolation; chosen over MongoDB for relational integrity (ADR-02)
Session & Cache	Redis	7.x	Session and OTP-service expiry, refresh token revocation, timed assessment sessions, and leaderboard sorted sets
Message Broker	RabbitMQ	3.12	Topic exchange (edura.events) for decoupled event publishing across 11 services
API Gateway / Proxy	Nginx	Alpine	Upstream routing, CORS headers, and 100 req/min rate limiting across all service paths
Container Orchestration	Docker Compose	—	Local full-stack environment for development; Kubernetes manifests maintained for cloud deployment
Cloud Platform	Microsoft Azure	—	AKS, ACR, and Azure Key Vault
Media & CDN	Cloudinary	—	Signed URL delivery (1-hour expiry) for course attachments and certificate assets
Video Streaming	YouTube (nocookie)	—	Secure embed tokens (youtube-nocookie.com) to prevent unauthorised access
Payment Gateway	PayHere	—	Sri Lanka-localised payment gateway; MD5-based webhook signature verification (as PayHere specifies); sandbox checkout URL
Email Delivery	Resend API	—	Transactional email for verification and reset links, payment confirmation, and certificates (updated from SMTP)
Linter (Backend)	Ruff + Black	—	PEP 8-compliant linting and formatting; config committed to pyproject.toml
Linter (Frontend)	ESLint + Prettier	—	Airbnb-style rules; config committed to .eslintrc.json and .prettierrc

Testing (Backend)	pytest	—	Unit and integration tests with SQLite in-memory database overrides
CI/CD	GitHub Actions	—	Parallel matrix test pipeline across all 11 services on every push

4.2 Module and Component Descriptions

The EDURA backend is composed of 11 independent FastAPI microservices, each with its own PostgreSQL database schema, Alembic migration history, and test suite. Services do not call one another over HTTP; they communicate only asynchronously through events on the RabbitMQ topic exchange (Section 3.3). The only outbound HTTP call is from the Notification Service to the Resend email API.

All services are exposed through Nginx at http://localhost/api/<service>/* and listen on dedicated ports (8001–8011) internally.

4.2.1 Auth Service (Port 8001)

The Auth Service manages all authentication and session lifecycle operations. It is the trust anchor for the entire platform.

Key responsibilities:

- User registration with email verification through a signed link (valid for 24 hours; a new link can be requested once every 60 seconds)
- Password-based login issuing 15-minute JWT access tokens and 7-day HttpOnly refresh token cookies
- JWT refresh token rotation with reuse detection (token theft mitigation)
- Password reset through an emailed single-use link valid for 60 minutes, which also revokes all sessions; an OTP service (one-time codes with a 5-minute Redis lifetime) exists but is not used by the user flows
- Google Sign-In: the frontend obtains a Google access token, and the Auth Service verifies it with Google and issues its own tokens

Database tables: users, refresh_tokens, otp_codes

Code excerpt — JWT access token creation:

```
# auth_service/services.py
class TokenService:
    @staticmethod
    def create_access_token(user_id: int, role: str, email: str | None = None) -> str:
        expire = datetime.now(timezone.utc) + timedelta(
            minutes=settings.ACCESS_TOKEN_EXPIRE_MINUTES
        )
        payload = {
```

```
        "sub": str(user_id),
        "role": role.upper(),
        "exp": expire,
        "iat": datetime.now(timezone.utc),
    }
    if email:
        payload["email"] = email
    return jwt.encode(payload, settings.SECRET_KEY, algorithm=settings.ALGORITHM)
```

Test coverage: 93% (unit + integration tests via test_auth_service.py, test_migrations.py, test_shared_messaging.py)

4.2.2 User Service (Port 8002)

The User Service manages user profile records that supplement the core authentication record held in the Auth Service. An account is created in the Auth Service on registration; the User Service maintains extended profile attributes (first name, last name, mobile number, date of birth).

Key responsibilities:

- Paginated user listing (admin only)
- Profile retrieval (own record or admin access)
- Profile update (own record or admin)
- Role assignment (admin only)
- Soft-delete of user accounts (admin only)

Shared security module — shared/auth.py:

```
# shared/auth.py — importable require_role dependency used by all 11 services
def require_role(roles: list[str]):
    async def dependency(token: str = Depends(oauth2_scheme)):
        payload = verify_jwt(token)
        if payload.get("role", "").lower() not in [r.lower() for r in roles]:
            raise HTTPException(status_code=403, detail="Insufficient permissions")
        return payload
    return dependency
```

This decorator is imported and used uniformly across all 11 services to enforce RBAC without duplication.

4.2.3 Course Service (Port 8003)

The Course Service implements the three-tier content hierarchy: Course → Module → Lesson. Teachers create and manage their own courses; administrators can manage all courses.

Key responsibilities:

- POST / — teacher creates a course (initial status: DRAFT)
- PUT /{course_id} — update; publishing requires ≥1 module with ≥1 lesson
- POST /{course_id}/modules/{module_id}/lessons — create lesson with youtube_video_id and cloudinary_asset_url
- Emits course.published RabbitMQ event on status change to PUBLISHED

Status lifecycle: DRAFT → PUBLISHED → ARCHIVED

Engineering decision: The assert_course_owner() helper was implemented to ensure teachers can only mutate their own courses while admins bypass the ownership check a clean separation avoiding duplication of role logic across endpoints.

4.2.4 Content Service (Port 8004)

The Content Service handles secure media delivery, ensuring that course content is only accessible to enrolled students.

Key responsibilities:

- POST /signed-url — generates Cloudinary signed URL with 1-hour expiry; enforces enrollment check for students
- GET /lessons/{lesson_id}/stream — returns YouTube nocookie embed parameters
- POST /upload — file upload with MIME type and magic-number byte signature validation (≤25 MB, PDF/PNG/JPEG only)

Enrollment access check:

```python
# content_service/enrollment_client.py
async def is_enrolled(user_id: int, course_id: int) -> bool:
    async with httpx.AsyncClient() as client:
        resp = await client.get(
            f"{ENROLLMENT_SERVICE_URL}/enrollments",
            params={"user_id": user_id, "course_id": course_id}
        )
        return resp.json().get("status") == "active"
```

Test coverage: 100% router coverage (20 tests in test_content.py)

4.2.5 Enrollment Service (Port 8005)

The Enrollment Service manages course access grants and revocations. It operates primarily as a RabbitMQ consumer rather than a request-driven REST API, providing clean decoupling from the payment workflow.

Key responsibilities:

- Consumes payment.success events → creates or extends ACTIVE enrollment by 30 days; publishes enrollment.activated

- Consumes subscription.expired events → updates enrollment status to SUSPENDED
- GET /api/enrollments — query endpoint for external access checks (used by Content Service)

Test coverage: 87% (test_enrollment.py)

4.2.6 Payment Service (Port 8006)

The Payment Service implements a hybrid payment model: automated PayHere gateway checkout for card and online payments, and a manual bank receipt upload path for bank transfers.

Key responsibilities:

- POST /checkout — initiates PayHere checkout session; generates the MD5-based hash PayHere requires for gateway authentication
- POST /webhook/payhere — verifies PayHere webhook signature; processes confirmed payments idempotently via unique transaction_id
- POST /receipts/upload — accepts PDF/JPEG/PNG bank receipt files; stores to cloud storage
- POST /receipts/{id}/verify — admin approves or rejects receipt; triggers payment event accordingly
- GET /admin/payments — admin paginated payment ledger

HMAC webhook verification:

```
# payment_service/utils.py
def verify_payhere_signature(merchant_id, order_id, amount,
                             currency, status_code, md5_sig) -> bool:
    secret_hash =
hashlib.md5(PAYHERE_MERCHANT_SECRET.encode()).hexdigest().upper()
    expected = hashlib.md5(
        f"{merchant_id}{order_id}{amount}{currency}{status_code}{secret_hash}".encode()
    ).hexdigest().upper()
    return hmac.compare_digest(expected, md5_sig.upper())
```

Engineering decision: Idempotency keys on transaction_id prevent double-charge scenarios under network timeouts, a critical correctness requirement identified during the OWASP A08 (Integrity Failures) review.

4.2.7 Assessment Service (Port 8007)

The Assessment Service provides a secure, timed quiz environment for MCQ and True/False assessments, with automated grading and anti-cheat mechanisms.

Key responsibilities:

- POST /api/assessments/{id}/start — initialises Redis-backed timed session with TTL equal to the quiz time limit; shuffles questions; obfuscates answer keys
- POST /api/assessments/{id}/submit — auto-grades MCQ and True/False; flags Short Answer for manual review; handles auto-submit on timer expiry; publishes assessment.graded event
- POST /api/assessments/{id}/violations — logs tab-switch and focus-loss anti-cheat events into ViolationLog

Test coverage: 84% (test_assessment.py)

4.2.8 Progress Service (Port 8008)

The Progress Service tracks student learning activity and maintains the leaderboard state.

Key responsibilities:

- Lesson watch status tracking and aggregation
- Course completion percentage calculation (Business Rule BR-004: completion when all lessons watched)
- Redis sorted-set leaderboard caching for real-time ranking without database load

4.2.9 Notification Service (Port 8009)

The Notification Service is a pure consumer. It has no REST endpoints exposed externally. It dispatches transactional emails in response to RabbitMQ events.

Consumed events and email templates:

Routing Key	Trigger	Email Template
auth.email-verification-requested	Registration / new verification link requested	email_verification.html
payment.success	Payment confirmed	payment_confirm.html
certificate.issued	Course completed	cert_issued.html
proctoring.violation	Anti-cheat violation detected	violation_alert.html
auth.password-reset-requested	Password reset requested	password_reset.html
auth.password-changed	Password changed	password_changed.html
auth.otp-requested	OTP endpoint (not used by the user flows)	otp_email.html

Email templates are rendered using Jinja2 and dispatched via the Resend API (updated from SMTP during Sprint 7). The service records all delivery outcomes, including failures, in a NotificationLog table.

Test coverage: 92% (test_templates.py, test_consumer.py, test_email_service.py)

4.2.10 Analytics Service (Port 8010)

The Analytics Service provides aggregate metrics and downloadable reports for administrators and teachers.

Key responsibilities:

- Revenue summary and per-course payment metrics
- Student engagement analytics (lesson completion rates, assessment score distributions)
- PDF report generation using ReportLab
- PDF progress report generation with ReportLab, emailed to the student (Excel export not implemented)
- Redis-cached query summaries for fast dashboard response times

4.2.11 Admin Service (Port 8011)

The Admin Service provides the administrative control plane for the system.

Key responsibilities:

- Manual bank receipt approval and rejection workflow
- User role assignment and account management
- Comprehensive audit logging for all administrative actions

4.3 Database Implementation

4.3.1 Database Architecture

Each of the 11 microservices has its own isolated PostgreSQL database schema on Neon (serverless managed PostgreSQL). Database isolation follows the database-per-service pattern from the SDS, ensuring that a schema change in one service cannot impact another.

The 11 databases are:

auth_db · user_db · course_db · content_db · enrollment_db · payment_db · assessment_db · progress_db · notification_db · analytics_db · admin_db

In the local Docker Compose environment, all 11 databases are initialised within a single PostgreSQL 16 container using a custom init-dbs.sh script. In the cloud environment, each service connects to its own Neon database via a per-service DATABASE_URL environment variable.

4.3.2 ORM Models

All database entities are defined using SQLAlchemy 2.0 mapped column syntax. The Base class is shared per-service and bound to the service's engine. An example from the Auth Service:

```
# auth_service/models.py
class RefreshToken(Base):
```

```
    __tablename__ = "refresh_tokens"

    id: Mapped[int] = mapped_column(Integer, primary_key=True, autoincrement=True)
    user_id: Mapped[int] = mapped_column(Integer, nullable=False)
    token_hash: Mapped[str] = mapped_column(String(512), nullable=False, unique=True)
    is_revoked: Mapped[bool] = mapped_column(Boolean, nullable=False, default=False)
    expires_at: Mapped[DateTime] = mapped_column(DateTime(timezone=True),
nullable=False)
    created_at: Mapped[DateTime] = mapped_column(
        DateTime(timezone=True), server_default=func.now(), nullable=False
    )
    __table_args__ = (Index("ix_refresh_tokens_user_id", "user_id"),)
```

Refresh tokens are stored as SHA-256 hashes — raw token values are never persisted in the database (OWASP A02 compliance).

4.3.3 Migrations

Database migrations are managed with Alembic, with one migration chain per service. The CI pipeline includes an automated pytest fixture that verifies the full upgrade → downgrade → upgrade cycle against a fresh test database on every push.

Engineering decision: At runtime, services call Base.metadata.create_all() on startup for lightweight schema bootstrapping during development. Production schema changes use Alembic migrations exclusively.

4.3.4 Connection Management

All services use SQLAlchemy's pool_pre_ping=True with SSL enforcement for Neon connections. The get_db() generator function provides session dependency injection via FastAPI's Depends() pattern, ensuring sessions are always closed after each request:

```
# shared pattern — database.py in each service
def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()
```

4.4 CI/CD Pipeline Description

The EDURA CI/CD pipeline is implemented using GitHub Actions and hosted in the edura-infra repository. It enforces quality gates on every push to develop or main, and on every pull request targeting those branches.

4.4.1 Backend CI (edura-backend)

Each of the 11 microservices is tested in a parallel matrix job.

Stage	Trigger	Actions
Lint & Format	Every push	Ruff (pycodestyle, pyflakes, isort, bugbear) + Black format check
Security Scan	Every push	pip-audit dependency vulnerability scan; fails on high/critical severity
Unit & Integration Tests	Every push	pytest with SQLite in-memory DB override; coverage report generated
Migration Verification	Every push	Alembic upgrade → downgrade → upgrade cycle on isolated test DB

4.4.2 Infrastructure CI (edura-infra)

Stage	Trigger	Actions
RabbitMQ Topology Test	Push to develop/main	Runs rabbitmq/setup.py twice to verify idempotency
Docker Smoke Test	Push to develop/main	Builds all Docker images; spins up full compose stack; verifies no container exits
Deploy – Staging	Merge to develop	Triggers staging deployment workflow
Deploy – Production	Merge to main (manual approval)	Triggers production deployment

4.4.3 Infrastructure as Code

The edura-infra repository contains:

- docker-compose.yml — full local development stack with 11 backend services, PostgreSQL 16, Redis 7, RabbitMQ 3.12-management, and Nginx
- k8s/staging/ — Kubernetes deployment manifests for Azure AKS staging environment
- nginx/nginx.conf — upstream routing config with CORS and rate limiting
- rabbitmq/setup.py — idempotent RabbitMQ topology provisioning script
- postgres/init-dbs.sh — PostgreSQL multi-database initialisation script

4.4.4 Secrets Management

All secrets (API keys, database passwords, JWT secret keys, PayHere credentials, Cloudinary credentials, Resend API keys) are stored as GitHub Secrets for CI/CD pipelines and as .env files in .gitignore for local development. A .env.example file with placeholder values is committed to each service repository to document required environment variables.

4.5 Engineering Challenges and Decisions Made

4.5.1 Cloud Cost Optimisation — AKS Deprovisioning (Sprint 5)

Challenge: After successfully deploying and verifying the full EDURA stack on Azure Kubernetes Service (AKS) during Sprint 5 (DDP-#18), the team identified that running a multi-node AKS cluster continuously would quickly exhaust the allocated Azure student credits.

Decision: The AKS cluster was gracefully deprovisioned after staging health was validated. All development and integration testing from Sprint 6 onwards was performed using the local Docker Compose environment, with Kubernetes manifests maintained in edura-infra/k8s/ for the final Sprint 8 deployment. This decision was endorsed by the supervisor and documented as RSK-04 in the Sprint 5 retrospective.

4.5.2 Python Dependency Standardisation (Sprint 5)

Challenge: CI pipeline failures occurred due to conflicting library versions. Specifically python-multipart requirements with FastAPI form handling, and deprecation warnings from python-jose.

Decision: Migrated all 11 services from python-jose to PyJWT for JWT operations. Pinned stable FastAPI and Pydantic v2 versions across all requirements.txt files. Ruff's B (bugbear) ruleset was enabled to catch similar future issues at lint time.

4.5.3 Payment Idempotency and Double-Charge Prevention (Sprint 6)

Challenge: Network timeouts during PayHere webhook delivery could cause the payment gateway to retry the same payment.success notification, risking double enrollment or double revenue recording.

Decision: All payment processing operations are guarded by a unique transaction_id idempotency key persisted in the payments table. Any webhook delivery for an already-processed transaction returns 200 OK without executing side effects. Documented as RSK-05.

4.5.4 Asynchronous Event Debugging (Sprint 6)

Challenge: Debugging asynchronous workflows across RabbitMQ consumers in multiple services (e.g., tracing a payment.success event through enrollment activation and notification dispatch) was initially difficult without centralised observability.

Decision: event chains are followed by reading the logs of each service in sequence. Structured JSON logging and correlation identifiers were considered but are not implemented in the code, so end-to-end tracing across services remains a manual task and is listed as future work.

4.5.5 Notification Service Email Provider Migration (Post Sprint 6)

Challenge: The original SMTP-based notification service was difficult to configure reliably in CI and staging environments, and delivered unreliable email in the Sri Lankan ISP context.

Decision: Migrated to the Resend API, which provides a managed transactional email service with reliable delivery. The SMTP_* configuration block was removed and replaced with RESEND_API_KEY and RESEND_FROM_EMAIL. This change was documented in CHANGELOG.md under [Unreleased].

4.5.6 Frontend–Backend Contract Alignment (Sprint 6)

Challenge: Initial TypeScript frontend integrations encountered property naming discrepancies. camelCase in TypeScript versus snake_case in Pydantic v2 schemas causing runtime contract failures.

Decision: Enforced OpenAPI schema exports (/openapi.json) from each FastAPI service and strict TypeScript interface generation from those exports before any new frontend feature development. Documented as RSK-10.

Chapter 5: Testing

This chapter describes how the EDURA system was verified. It sets out the testing strategy, the test levels and tools used, the endpoint-level test case register, the coverage achieved, the cross-service integration and smoke tests, and the performance testing status. Results are reported as measured; where a planned activity was not completed, this is stated explicitly and tracked in Section 6.3.

5.1 Testing Strategy

The strategy follows the test pyramid defined in the SENG 34213 development guideline (§6.2). Testing is part of the Definition of Done for every issue, so each service was tested by the developer who owned it and was gated by the CI pipeline described in Section 4.4 before merging to develop or main.

Level	Scope	Target (guideline §6.4)	Tooling
Unit	Single function or class in isolation (services, validators, security helpers)	80% of new code	pytest, coverage.py
Service integration	Real HTTP requests to each FastAPI app against a test database, with external systems mocked	100% of API endpoints have at least one test	pytest, FastAPI TestClient, SQLite test database override
Cross-service integration	Multi-service flows through the gateway (e.g. payment to enrollment to notification), run against the deployed staging environment or a throwaway local stack	Critical flows	pytest, httpx (edura-tests/integration)
End-to-end	Critical user journeys on staging: an API-level smoke script after each deploy, and three browser (UI) flows	Top critical flows	Python smoke script (edura-tests/smoke), Playwright (edura-tests/e2e)
Static and security	Style, bugs and dependency vulnerabilities	No high or critical findings	Ruff, Black, pip-audit, audit-ci, hadolint, yamllint, shellcheck
Performance	Response time and concurrent load on a deployed service	95% of interactions under 2.0 s (NFR-001); 500 concurrent users (NFR-009)	Python load script (edura-tests/performance), asyncio and httpx

Tests follow the Arrange-Act-Assert pattern with descriptive names. External dependencies (PayHere, Cloudinary, Resend, YouTube and message publishing) are replaced with mocks so that tests are deterministic and run in CI without credentials. Negative paths are tested alongside happy paths: invalid input, missing or expired tokens, role violations (a student calling a teacher or admin endpoint), ownership violations, duplicate submissions and repeated webhook deliveries.

5.2 Test Environment

- Service tests run with an isolated SQLite database injected through the FastAPI database dependency override, so no PostgreSQL instance is required and every test starts from a clean schema.
- Migration verification runs an Alembic upgrade, downgrade and upgrade cycle on an isolated database in CI.
- Cross-service integration tests (203 tests) call the gateway over HTTP. Those for every service except payments and enrolments ran against the deployed staging environment (https://staging.eduralms.com). The payment and enrolment tests, whose webhook checks need the PayHere credentials, ran against an in-process mock gateway in CI and against a throwaway local stack (the 11 services, a new PostgreSQL 16 database, Redis 7, RabbitMQ 3.12 and Nginx), so that no test data was left in the shared staging database.
- Each backend service runs in its own parallel job in the GitHub Actions matrix. The infrastructure pipeline additionally runs the RabbitMQ topology idempotency test and a Docker smoke test.
- Browser tests (Playwright, Chromium) run against staging and remove the accounts and enrolments they create through the admin API. Performance tests run from one client through the gateway and from a short-lived pod inside the cluster.

5.3 Test Execution Summary

5.3.1 Service-Level Tests

The table lists the number of automated test functions in each service test suite, the number of gateway endpoints each service exposes and the result of the last recorded run. Endpoints are counted as unique HTTP method and gateway path pairs, as recorded in the endpoint register.

Service	Test functions	Endpoints	Passed in last run
admin_service	66	14	66
analytics_service	97	10	97
assessment_service	141	34	140 (1 expected failure)
auth_service	88	20	88
content_service	37	7	37

course_service	81	25	81
enrollment_service	65	19	65
notification_service	38	2	38
payment_service	44	14	44
progress_service	36	9	36
user_service	85	20	85
Total	778	174	777 (1 expected failure)

In total the 11 service suites contain 778 automated test functions and 174 gateway endpoints (Section 5.4 shows how many endpoints are matched to a test). All of them passed in the recorded run of 28 October 2026 except one expected failure (xfail) in assessment_service. The run is recorded in edura-documents/testing/test-results.md.

5.3.2 Cross-Service Integration and Smoke Tests

The edura-tests repository holds 203 cross-service integration tests organised by business flow: authentication, users, courses, content, payment-to-enrollment, assessment, progress, notification and analytics, and admin. 198 of them ran against staging on 28 October 2026 (197 passed, 1 skipped); the 5 payment-to-enrollment tests ran against an in-process mock gateway and passed. The payment-to-enrollment flow verifies the event chain from a verified payment, through enrollment activation, to content access. A separate smoke script (critical_path.py) walks the critical path as a real user would: it registers and verifies an account, signs a PayHere webhook with PayHere's MD5-based signature, confirms enrollment, starts and submits a timed assessment, and removes the data it created. Six Playwright browser tests cover three critical flows (register and sign in; browse and enrol; take an exam and see the result) and all passed against staging. A further 68 payment and enrolment tests proposed in a separate pull request passed against the throwaway local stack.

5.4 Test Case Register

In line with guideline §6.3.3 and §6.4, every test case is listed in the Test Case Register (edura-documents/testing/test-register.md, Appendix B) with its type, priority, test file and the status from the recorded run (IDs TC-0001 to TC-1850), and every API endpoint is tracked in an Endpoint Register (testing/endpoint-register.md) that shows which tests call it. The status of each test comes from the run recorded in testing/test-results.md.

Measure	Value
Test cases in the register	1,850
Endpoints (unique method and gateway path)	174

Endpoints matched to at least one test	156 confirmed, 17 probable (static analysis)
Endpoints with no test found	1
Result of the recorded run	1,836 passed, 1 failed (since fixed), 5 skipped or expected failures, 8 not run

A representative test case, in the documentation format required by the guideline, is shown below. The full table of 29 key test cases, mapped to the functional requirements, is in edura-documents/testing/test-case-table.md.

Field	Content
Test Case ID	TC-FR-01-02
Test Case Name	Wrong password is refused
Related Requirement	FR-001 (SDS numbering: authentication and role-based access)
Related Test File	edura-tests/integration/test_auth_flow.py (test_given_wrong_password_when_login_then_401)
Type / Priority	Integration / P1 - Must pass
Preconditions	A verified student exists
Input	POST /api/auth/login with the student's email and a wrong password
Expected Output	HTTP 401 and no tokens
Actual Output	HTTP 401 (staging, 28 October 2026)
Status	Pass

5.5 Coverage Report Summary

Coverage is generated in the CI test stage: coverage.py (lines and branches, production code only; test files and migrations are excluded) for each backend service and Jest coverage for the Next.js frontend. The reports are published as CI artefacts: Backend CI run 40655783476 and Frontend CI run 40653950730 on the develop branch. Every service is above the 80% target, and the critical business logic (auth_service and payment_service) is above the 90% branch target.

Service / component	Coverage (lines / branches)	Target
admin_service	97.9% / 100.0%	80%
analytics_service	98.7% / 99.1%	80%
assessment_service	99.2% / 97.1%	80%
auth_service	98.4% / 97.5%	80% (90% branch, critical logic)

content_service	98.5% / 100.0%	80%
course_service	99.1% / 96.7%	80%
enrollment_service	98.8% / 97.2%	80%
notification_service	95.7% / 96.7%	80%
payment_service	99.0% / 97.4%	80% (90% branch, critical logic)
progress_service	97.8% / 97.6%	80%
user_service	98.6% / 100.0%	80%
Backend, all services	98.6% / 97.8%	80%
Frontend (Next.js)	98.3% lines / 91.5% branches (97.0% functions)	80%

Source: the coverage figures are taken from the CI coverage artefacts of the runs named above (edura-documents/testing/coverage-report.md). They were not estimated by hand.

Frontend verification combines component tests for the main student, teacher and admin screens with the cross-service and smoke tests, which exercise the same APIs the interface calls. Manual checks covered the quiz player, checkout flow and admin panel. In Sprint 8, Playwright browser tests were added for the three critical flows.

5.6 Performance Test Results

Performance was measured with a Python load script (edura-tests/performance/load_test.py, asyncio and httpx) that sends read-only GET requests. Response time (NFR-001: 95% of interactions in under 2.0 seconds) was measured through the public gateway at about 75 requests per minute, below the gateway's limit of 100 requests per minute per client address. Concurrency (NFR-009: at least 500 concurrent sessions) was measured with a short-lived load pod inside the cluster that called the course service directly, because one client address cannot create real load through the gateway. The runs were made on 28 October 2026 against staging, which runs one replica per service on a small, low-cost cluster. About 240 ms of every gateway response is network travel from Sri Lanka to Azure East US.

Scenario	Concurrent users	Median (ms)	p95 (ms)	Error rate	Result
Course list (gateway)	1 (paced)	247	1,816	0%	Pass (under 2 s)
Course detail (gateway)	1 (paced)	240	325	0%	Pass (under 2 s)

Platform statistics (gateway)	1 (paced)	246	319	0%	Pass (under 2 s)
Class packages (gateway)	1 (paced)	246	326	0%	Pass (under 2 s)
Course service, direct	10	44	155	0%	Pass
Course service, direct	20	94	443	0%	Pass
Course service, direct	40	236	10,003	10.7% (timeouts)	Fail

NFR-001 (response time) is met for the endpoints tested: typical responses take about 250 ms from Sri Lanka, and the slowest case, the course list, had a 95th percentile of 1.8 s, just under the 2.0 s target. NFR-009 (500 concurrent users) is not met on the staging hardware: one service handles about 20 concurrent clients without errors and begins to time out at 40. In the first load runs an overloaded course service stopped answering and had to be restarted by hand. After health probes and a second course-service replica were added (DDP-#73), an overloaded pod was restarted automatically by its liveness probe and the public API stayed available. These results come from one client, read-only public endpoints and low-cost staging hardware, so they are a lower bound and not a prediction for production; sign-in, exam submission and payment were not load tested. The full results are in edura-documents/testing/performance-report.md, and the capacity limit is recorded as LIM-06 in Section 6.3.

5.7 Defects Found and Test Limitations

- DEF-01 (concurrent refresh requests returning 409) is not fixed: the Auth Service has no lock on refresh-token rotation, so it remains a known defect (Section 6.3).
- Payment idempotency (RSK-05) is covered by tests that replay the same webhook and confirm that no duplicate enrollment or revenue record results.
- Mobile responsiveness (LIM-05) and cross-browser behaviour were not formally tested; the Playwright tests run in Chromium only.
- Remaining limitations: automated tests do not call live PayHere, Cloudinary or Resend services, so provider-side behaviour is verified only through mocks and manual staging checks. The payment webhook tests ran against an in-process mock gateway and a throwaway local stack, not staging, because the staging PayHere credentials were not available. No formal usability study with ICT – Dimuthu students was carried out.
- Findings fixed in Sprint 8: a gateway route test for the enrolment service had become out of date and was not run in CI (fixed, DDP-#72); the course service stopped answering under load (health probes and a second replica added, DDP-

#73). One analytics test failed once and passed on three reruns and has not been investigated.

5.8 Security Compliance Evidence

The OWASP Top 10 review (edura-documents/security/owasp-checklist.md, updated 28 October 2026) compares the code, configuration and CI workflows with guideline §8.1. The security response headers were checked on the deployed staging site. A STRIDE threat model is in edura-documents/security/threat-model.md; it is to be added to the SDS security section.

Risk	Status	Evidence and gaps
A01 Broken Access Control	Met	132 of 174 endpoints require a token or role; the other 42 are public by design and were reviewed; 403 and 401 responses are asserted in tests.
A02 Cryptographic Failures	Partly met	bcrypt (cost 12), hashed refresh tokens, HTTPS; a fallback signing key exists in the code (the staging key was checked and differs).
A03 Injection	Met	ORM and validated schemas; no injection-payload test.
A04 Insecure Design	Partly met	Threat model written; it still has to be added to the SDS.
A05 Security Misconfiguration	Partly met	Security headers, rate limiting, default RabbitMQ account removed; no Content-Security-Policy.
A06 Vulnerable Components	Partly met	pip-audit and audit-ci in CI; 22 accepted frontend advisories documented (ADR-14).
A07 Authentication Failures	Partly met	Refresh-token rotation and short-lived access tokens; no account lockout.
A08 Integrity Failures	Partly met	Images tagged by commit, npm lock file; no Python lock file.
A09 Logging Failures	Not met	Authentication events are not logged; recipient emails appear in notification logs.
A10 Server-Side Request Forgery	Met	Outbound calls go only to fixed addresses.

The known security limitations are accepted and recorded in ADR-17. They are not fixed in this release, to avoid changing the running system before the demonstration.

Chapter 6: Evaluation

This chapter evaluates the EDURA system against the original objectives stated in the Software Requirements Specification (SRS), documents the limitations and known defects identified during development, and outlines recommended future work.

6.1 Degree of Objectives Met

The following table cross-references each primary functional objective with its implementation status at the time of the final demonstration on the staging environment. Objective IDs follow this report; the SDS numbers the functional requirements FR-001 to FR-015.

Objective ID	SRS Requirement	Implementation Status	Notes
FR-01	User Registration and Email Verification	Fully Implemented	Email verification link (signed, time-limited token) on registration; one-time codes available through the OTP endpoints; 98.4% line coverage in auth_service
FR-02	Role-Based Access Control (Student / Teacher / Admin)	Fully Implemented	Centralised role check (shared/auth.py) used across all 11 services; 132 of 174 endpoints require a token or role, and the other 42 are public by design (OWASP review)
FR-03	OAuth2 / Google Sign-In	Fully Implemented	Google sign-in: the frontend obtains a Google access token and the Auth Service verifies it with Google and issues its own HS256 tokens (Asgardeo was not used)
FR-04	Course, Module, and Lesson Management (Teacher)	Fully Implemented	Full CRUD with DRAFT/PUBLISHED/ARCHIVED lifecycle; teacher ownership enforcement
FR-05	Secure Media Delivery (Video + Attachments)	Fully Implemented	YouTube nocookie embeds + Cloudinary signed URLs (1-hour expiry); MIME/magic-number validation
FR-06	Automated Assessment with MCQ / True-False Grading	Fully Implemented	Redis timed sessions, auto-grading, manual review for Short Answer
FR-07	Anti-Cheat Tab-Switch Detection	Fully Implemented	ViolationLog records tab-switch and focus-loss events during assessments

FR-08	Hybrid Payment Processing (PayHere + Bank Receipt)	Partly Implemented	PayHere webhook signature (MD5, as PayHere specifies), idempotent handling and bank receipt upload with admin verification are implemented; the PayHere option is hidden in the checkout page and uses PayHere's sandbox, so the bank deposit is the active payment method
FR-09	Automated Enrollment Lifecycle	Fully Implemented	RabbitMQ-driven enrollment grant and suspension; 98.8% line coverage in enrollment_service
FR-10	Progress Tracking and Course Completion	Fully Implemented	Lesson watch status, completion percentage, leaderboard via Redis sorted sets
FR-11	Transactional Notification Emails	Fully Implemented	Verification and password-reset links, payment confirmation, certificate and violation emails via the Resend API; 95.7% line coverage in notification_service
FR-12	Admin Dashboard and Receipt Verification	Fully Implemented	Receipt approval/rejection endpoints with audit trail
FR-13	Analytics Reports (PDF / Excel)	Partly Implemented	PDF progress reports (ReportLab) are generated and emailed to the student; the Excel export was not implemented; summary queries are cached in Redis
FR-14	Student Progress Dashboard (Frontend)	Fully Implemented	Backend and Next.js student dashboard delivered
FR-15	Course Content Viewer (Frontend)	Fully Implemented	Backend and Next.js viewer with YouTube embed delivered
FR-16	Assessment Quiz Player (Frontend)	Fully Implemented	Backend and Next.js quiz player with timer and anti-cheat violation recording delivered
FR-17	Teacher Course Builder UI (Frontend)	Fully Implemented	Course, module and lesson management screens delivered in the admin panel
FR-18	Payment Checkout UI (Frontend)	Partly Implemented	Checkout page with bank slip upload delivered; the PayHere redirect option is hidden in the page (sandbox)

FR-19	Admin Panel UI (Frontend)	Fully Implemented	Admin panel screens for payments, receipt verification and analytics delivered
NFR-01	Unit test coverage ≥ 80% on new code	Met	Backend 98.6% lines and 97.8% branches; frontend 98.3% lines and 91.5% branches (CI runs on develop)
NFR-02	95% of standard interactions processed in under 2.0 seconds (SRS NFR-001)	Met	Met for the endpoints tested: p95 below 2 s (course list 1.8 s); see Section 5.6
NFR-03	HTTPS / TLS for all external traffic	Fully Implemented	Caddy terminates HTTPS with an automatic Let's Encrypt certificate; HSTS and other security headers are set
NFR-04	OWASP Top 10 compliance	Partly met	A01, A03 and A10 met; A02, A04 to A08 partly met; A09 not met; no account lockout (A07); see Section 5.8
NFR-09	At least 500 concurrent sessions with no more than 20% degradation (SRS NFR-009)	Not met	About 20 concurrent clients per service on the low-cost staging cluster; see Section 5.6 and LIM-06

6.2 Client Feedback

A formal client demonstration session was conducted with Dimuthu Prabath (ICT – Dimuthu) following the design phase in SENG 31242. The client's feedback directly shaped several implementation decisions:

- The hybrid payment system (PayHere + manual bank receipt) was specifically requested to accommodate students who do not have access to card payments. The bank receipt flow is implemented and is the active payment method; the PayHere flow is implemented in the backend against PayHere's sandbox and is hidden in the checkout page for now.
- The leaderboard and gamification features were highlighted as high-priority to drive student engagement and retention. Implemented via Redis sorted sets in the Progress Service and prominently featured in the student dashboard.
- The client requested that teacher dashboards clearly distinguish between enrolled and visiting students for each course. Implemented via the enrollment access check in the Content and Course Services.

- A per-lesson notes download feature was requested during the feedback session. Included in the Content Service attachment delivery via Cloudinary signed URLs.

A formal client sign-off for the implemented system will be conducted during the Sprint 8 final panel demonstration.

6.3 Limitations and Known Defects

The following limitations and known defects have been identified and are tracked on the GitHub Project board:

ID	Description	Severity	Status
LIM-01	AI assistance feature (originally planned in the SRS) was descoped due to time constraints and API cost considerations. This feature is documented as future work.	Medium	Accepted (Descoped)
LIM-02	Analytics Service aggregation queries run directly against the transactional database in some edge cases; Redis caching covers the primary dashboard queries but not all ad-hoc report endpoints.	Low	Open (deferred to future work)
LIM-03	Short Answer question type requires manual teacher review. No AI-assisted grading is implemented (descoped alongside LIM-01).	Low	Accepted (Descoped)
LIM-04	The Notification Service dead-letter exchange (edura.dlx) and exponential backoff retry are configured but not load-tested under high queue volume.	Medium	Open (deferred to future work)
LIM-05	Mobile responsiveness of the Next.js frontend has not been formally tested on small-screen devices.	Low	Open (deferred to future work)
LIM-06	Performance testing was run on 28 October 2026 (Section 5.6). The response-time target (NFR-001) is met, but the target of 500 concurrent users (NFR-009) is not: one service handles about 20 concurrent clients on the low-cost staging cluster and times out at 40.	High	Open; accepted for this release, recorded in ADR-17 and Section 5.6

DEF-01	Concurrent refresh token requests from the same session occasionally result in a 409 race condition rather than a clean rotation. The Auth Service has no distributed lock on the rotate-and-invalidate step.	Medium	Open; not fixed
LIM-07	Excel export of analytics reports (SRS FR-012) is not implemented; only PDF reports exist.	Medium	Accepted (not delivered)
LIM-08	Completion certificates (SRS FR-015): the eligibility rule, the certificate record and the certificate.issued event work, but no certificate PDF is generated and the frontend has no certificate page.	Medium	Accepted (partly delivered)
LIM-09	Security limitations recorded in ADR-17: no account lockout, authentication events not logged, recipient emails in logs, a fallback signing key in the code, no Python lock file, no Content-Security-Policy and no Kubernetes network policies.	Medium	Accepted until after the demonstration
LIM-10	The PayHere payment option is hidden in the checkout page and uses PayHere's sandbox; only the bank deposit flow is active.	Medium	Accepted

6.4 Future Work

The following enhancements are recommended for development cycles following the SENG 34213 project:

1. AI-Assisted Features: The original SRS included an AI assistant for students (learning guidance, question explanation) and teachers (quiz generation from lesson content). This was descoped during Sprint 5 due to API cost constraints and time pressure. Integration with a hosted LLM API (e.g., Gemini API) is the recommended approach for a future sprint.
2. Mobile Application: A React Native or progressive web app (PWA) version of the frontend would expand reach to students using mobile devices, which is the dominant access device for the target demographic in Sri Lanka.

3. Live Class Sessions: Integration with a video conferencing API (e.g., Zoom SDK, Jitsi) to support live online tutoring sessions would complement the existing pre-recorded content delivery model.
4. Advanced Analytics and Predictive Insights: Machine learning models trained on assessment and progress data could identify at-risk students early and recommend targeted intervention, aligning with the institute's goal of improving student outcomes.
5. Automated Certificate Generation: when a student completes a course and passes every assessment, a certificate record with a placeholder link is stored and a certificate.issued event is published. A dedicated service generating PDF certificates with digital signatures, and a page where students can view and download them, would provide a verifiable credential for students.
6. Multi-Institute Support: Extending the platform from a single-tenant B2B solution to a multi-tenant SaaS platform would significantly expand the addressable market. This would require schema-level tenant isolation and per-tenant billing.
7. Distributed Lock for Token Rotation (DEF-01 Fix): Implementing a Redis-based distributed lock (using Redlock or a SET NX EX pattern) on the refresh token rotation operation would eliminate the race condition identified in DEF-01.
8. Excel Export and Security Hardening: implementing the Excel report export (FR-012), account lockout, authentication-event logging with masked email addresses, removal of the fallback signing key, pinned Python dependencies, a Content-Security-Policy and Kubernetes network policies (ADR-17 and the threat model).
9. Capacity: running more replicas on a larger node, tuning the connection pool and worker count, and caching or paging the course list would raise the number of concurrent users each service can handle (NFR-009).

Chapter 7: Conclusion

The EDURA Learning Management System has been successfully developed across four development sprints (Sprints 5–8) of the SENG 34213 System Development Project. The project directly implements the design artefacts produced in SENG 31242, translating the approved Software Requirements Specification and Software Design Specification into a working, tested, and deployable software product.

At the end of Sprint 8, the platform runs on a staging environment on Azure Kubernetes Service: all 11 FastAPI microservices and the Next.js frontend are deployed behind a Caddy and Nginx edge and expose 174 REST API endpoints, backed by 778 service tests with 98.6% line coverage and 97.8% branch coverage. The event-driven messaging topology on RabbitMQ has been verified end-to-end, confirming that payment, enrollment, progress, and notification workflows operate correctly in an asynchronous, decoupled manner. The CI/CD pipeline enforces code quality gates (linting, security scanning, and test coverage) on every push to develop and main, and deploys develop to staging automatically. The system does not yet meet every target: the 500-concurrent-user target is not met on the low-cost staging hardware, and the Excel export, the certificate PDF and several security measures are not delivered (Section 6.3).

The development process validated several key engineering principles in practice. The microservices architecture allowed four developers to work in parallel across discrete service boundaries without integration conflicts a direct benefit of the decoupled design established in the SDS. The database-per-service pattern, while adding initial complexity to the Alembic migration setup, provided the service isolation necessary for independent deployment. The shared require_role RBAC module demonstrated the value of a well-designed shared library: once implemented in Sprint 5, it was immediately consumed by all downstream services in Sprint 6 with zero additional RBAC engineering effort.

Two significant engineering challenges shaped the trajectory of the project: the cloud cost optimisation decision in Sprint 5 (deprovisioning the AKS cluster after verification and shifting to Docker Compose for local development) and the payment idempotency design for PayHere webhooks. Both challenges were resolved through deliberate engineering decisions documented in retrospectives and risk logs, demonstrating the team's ability to respond pragmatically to constraints without compromising product quality.

In conclusion, EDURA represents a significant modernisation of the operations of ICT – Dimuthu. The system delivers on the core promise of the original proposal: a centralised, secure, and scalable digital platform that replaces manual educational administration with structured, automated workflows — ultimately enabling the institute to serve more A/L ICT students more effectively across Sri Lanka.

Chapter 8: References

[1] E. F. Codd, "A relational model of data for large shared data banks," Commun. ACM, vol. 13, no. 6, pp. 377–387, Jun. 1970. https://doi.org/10.1145/362384.362685

[2] E. Evans, Domain-Driven Design: Tackling Complexity in the Heart of Software. Boston, MA, USA: Addison-Wesley, 2003.

[3] D. Hardt, Ed., "The OAuth 2.0 Authorization Framework," IETF, RFC 6749, Oct. 2012. [Online]. Available: https://datatracker.ietf.org/doc/html/rfc6749

[4] N. Sakimura, J. Bradley, M. Jones, B. de Medeiros, and C. Mortimore, "OpenID Connect Core 1.0 incorporating errata set 1," OpenID Foundation, Nov. 2014. [Online]. Available: https://openid.net/specs/openid-connect-core-1_0.html

[5] M. Jones, J. Bradley, and N. Sakimura, "JSON Web Token (JWT)," IETF, RFC 7519, May 2015. [Online]. Available: https://datatracker.ietf.org/doc/html/rfc7519

[6] OWASP Foundation, "OWASP Application Security Verification Standard (ASVS)," version 4.0, 2019. [Online]. Available: https://owasp.org/www-project-application-security-verification-standard/

[7] M. Kleppmann, Designing Data-Intensive Applications. Sebastopol, CA, USA: O'Reilly Media, 2017.

[8] C. Richardson, Microservices Patterns: With Examples in Java. Shelter Island, NY, USA: Manning Publications, 2018.

[9] S. Newman, Building Microservices: Designing Fine-Grained Systems, 2nd ed. Sebastopol, CA, USA: O'Reilly Media, 2021.

[10] R. T. Fielding, "Architectural Styles and the Design of Network-based Software Architectures," Ph.D. dissertation, Dept. of Information and Computer Science, Univ. of California, Irvine, 2000.

[11] G. Pan, Y. Li, and X. Zhang, "Digital transformation of educational institutions: A systematic review," Computers & Education, vol. 198, p. 104776, 2024. https://doi.org/10.1016/j.compedu.2023.104776

[12] Liliasari and L. S. Riza, "Learning management systems in higher education: A systematic literature review," International Journal of Emerging Technologies in Learning (iJET), vol. 18, no. 4, pp. 56–72, 2023.

[13] G. Zeng, "Gamification in education: A systematic review of effects on student motivation and achievement," Journal of Educational Technology & Society, vol. 27, no. 1, pp. 1–18, 2024.

[14] B. Nguyen-Viet, T. T. Le, and N. H. Pham, "Effects of gamification on student learning outcomes in online education environments," Education and Information Technologies, vol. 29, pp. 3401–3430, 2024. https://doi.org/10.1007/s10639-023-12058-3

Appendices A : GitHub Repository Submission

URL of the GitHub Organisation - https://github.com/Edura-LMS-Zynaptrix

URL(s) of all repositories -

- Edura-backend - https://github.com/Edura-LMS-Zynaptrix/edura-backend
- Edura-frontend - https://github.com/Edura-LMS-Zynaptrix/edura-frontend
- Edura-infra - https://github.com/Edura-LMS-Zynaptrix/edura-infra
- Edura-documents - https://github.com/Edura-LMS-Zynaptrix/edura-documents
- Edura-tests - https://github.com/Edura-LMS-Zynaptrix/edura-tests

URL of the deployed staging application – staging.eduralms.com

URL of the GitHub Project board - https://github.com/orgs/Edura-LMS-Zynaptrix/projects/1

Appendix B : Test case register

Test Case Register URL - https://github.com/Edura-LMS-Zynaptrix/edura-documents/tree/main/testing